TELONE CENTRE FOR LEARNING

Department of Telecommunications Engineering

Enhancing Workforce Management at Infor Relay Systems (Pvt) Ltd Through Implementation of a Real-Time RFID Employee Attendance System

A project report submitted in partial fulfilment of the requirements for the National Diploma in Telecommunications Engineering

(Qualification level to be confirmed against the student's official programme handbook)

Student Name: Dennis Kunakah

Registration Number: T2329469T

Programme: Telecommunications Engineering

Host Organisation: Infor Relay Systems (Pvt) Ltd

Supervisor: *[Supervisor's Name]*

August 2026

Declaration

I, Dennis Kunakah, declare that this project report is my own original work, carried out during my industrial attachment at Infor Relay Systems (Pvt) Ltd, and that it has not been submitted before for any other qualification. Where I have drawn on the work of others, this has been fully acknowledged through in-text citation and referencing.

Signature: _____

Date: _____

Supervisor's approval

Signature: _____

Date: _____

Acknowledgements

I would like to thank the management and staff of Infor Relay Systems (Pvt) Ltd for hosting my industrial attachment and for giving me the opportunity to identify a real workplace problem and build a working solution for it. Particular thanks go to my workplace supervisor and colleagues in the technical department, who supported the installation and testing of the attendance system on-site.

I am also grateful to my supervisor and the lecturers at TelOne Centre for Learning, whose guidance shaped both the technical direction of this project and the way it is presented in this report. Finally, I thank my family and friends for their patience and encouragement throughout the attachment period.

Contents

Declaration i

Acknowledgements ii

1 Introduction 1

1.1 Abstract . . . 1
1.2 Background of the Study . . . 1
1.3 Problem Statement . . . 2
1.4 Aim . . . 3
1.5 Objectives . . . 3
1.6 Proposed Solution . . . 3
1.7 Justification of the Study . . . 4
1.8 Scope and Limitations . . . 4
1.9 Structure of the Report . . . 4

2 Literature Review 5

2.1 Introduction . . . 5
2.2 Fundamentals of RFID Technology . . . 5
2.3 RFID-Based Attendance Systems: Related Work . . . 6
2.4 RFID Versus Biometric and Other Identification Methods . . . 7
2.5 Manual Attendance, Time Theft, and the Case for Automation . . . 7
2.6 Offline-First Design for Embedded and IoT Devices . . . 8
2.7 Local Service Discovery and Web Protocols . . . 8
2.8 Software Stack for the Server-Side Application . . . 9
2.9 Technology Adoption in the Local Context . . . 9
2.10 Summary and Research Gap . . . 9

3 Methodology 11

3.1 Introduction . . . 11
3.2 System Architecture . . . 11
3.3 Requirements Analysis . . . 12
3.3.1 Functional Requirements . . . 12
3.3.2 Non-Functional Requirements . . . 13
3.4 Hardware Design . . . 13
3.4.1 Component Selection . . . 13

3.4.2 Circuit Design 18
3.4.3 PCB Design 18
3.5 Firmware Design 19
3.6 Software Architecture 20
3.7 Database Design 21
3.8 Bill of Quantities 22
3.9 Testing Approach 23

4 Results 25
4.1 Introduction 25
4.2 The Finished Hardware 25
4.3 The Dashboard Application 26
4.3.1 Events 26
4.3.2 Student Registration 26
4.3.3 Live Attendance Dashboard 27
4.4 Evaluation Against Objectives 28
4.4.1 Objective 1: Automating Attendance Tracking 28
4.4.2 Objective 2: Ensuring Accurate Labour Costing 28
4.4.3 Objective 3: Integrating RFID Attendance with Existing Systems 28
4.5 Discussion 29

5 Conclusion and Recommendations 30
5.1 Conclusion 30
5.2 Recommendations 30
5.3 Project Timeline 31

A ESP32 Firmware Source Code 33

B ESP32 Pinout and Power Budget 45

C PC Application API Reference 48

References 50

List of Figures

3.1 High-level system architecture: the ESP32 reader station always initiates contact with the PC application, never the other way around. 12
3.2 ESP32-WROOM-32 development board used as the reader station's microcontroller 14
3.3 MFRC522 RFID reader module 15
3.4 16x2 character LCD with I2C (PCF8574) backpack 15
3.5 Active buzzer used for audible tap feedback 16
3.6 5 mm LED, used for the Wi-Fi and App status indicators 16
3.7 Power supply components: a mains adapter feeding an adjustable buck converter 17
3.8 Circuit schematic for the reader station, drawn in Proteus Design Suite . 18
3.9 PCB layout (top copper in blue, bottom copper in purple) for the reader station 19
3.10 3D render of the assembled reader station PCB 19
3.11 Tap-handling logic, shared identically between the live `/api/tap` path and the offline queue replayed through `/api/sync`. The main decision spine runs top to bottom; each outcome branches right to the resulting beep/LCD feedback and database action. 20
3.12 Entity-relationship diagram of the Prisma/SQLite data model. UnknownTap has no foreign key to Student because, by definition, no matching student exists yet. 22

4.1 The completed reader station, enclosed and ready for deployment at the entrance 25
4.2 Events screen: creating and monitoring the active attendance event . . . 26
4.3 Students screen: registering a card UID against a name and registration number 27
4.4 Live dashboard for the active event, showing computed Present / Left / Absent status per student 27

5.1 Project Gantt chart, May to July, with the literature review spanning the full project duration 32

List of Tables

3.1 Core hardware components 14
3.2 API endpoints exposed by the PC application 21
3.3 Bill of Quantities for one reader station 23

B.1 ESP32-WROOM-32 pin assignments 46

C.1 Full API route listing 48

Chapter 1

Introduction

1.1 Abstract

Infor Relay Systems (Pvt) Ltd, like many organisations, has relied on a paper attendance register and supervisor sign-off to track when employees arrive and leave work. This method sounds simple, but in practice it creates more problems than it solves. Registers get filled in late, entries go missing, and it is genuinely hard to prove exactly when someone clocked in on a busy morning. Those small gaps add up: payroll staff spend hours reconciling hand-written times, and the business loses confidence in the numbers it is paying people against.

This project set out to replace that register with something that records the moment automatically. The result is a Real-Time RFID Employee Attendance System built around an ESP32 microcontroller, an MFRC522 RFID reader, and a purpose-built Next.js web application running on a PC at the front desk. Each employee is issued an RFID card. Tapping it at the reader on the way in and again on the way out is all that is required — the station beeps, the LCD confirms the name, and the PC dashboard updates within seconds to show who is *Present*, who has *Left*, and who is *Absent* for the day.

What makes the design worth documenting is not the RFID reader itself — that part is well understood — but the fact that the reader station keeps working when the network does not. Every tap is written to the device's own flash storage first; forwarding it live to the server is a bonus, not a requirement. If Wi-Fi drops or the PC is rebooted, taps queue locally and drain automatically once the connection returns. Two indicator LEDs make the difference between "no Wi-Fi" and "Wi-Fi is fine but the server is down" visible to whoever is on the door, instead of leaving them guessing. This report walks through why that design was chosen, how it was built and wired, what it looks like running, and how well it met the objectives set out at the start of the attachment.

1.2 Background of the Study

Attendance tracking sits at the intersection of two things every organisation cares about: paying people correctly and knowing who is on site. At Infor Relay Systems (Pvt) Ltd,

both of those depend on a manual register signed at a supervisor's desk. It is a method that has worked, in a loose sense, for a long time — but it was never built to survive a busy shift change, a supervisor who is away, or a payroll clerk who has to turn scrawled timestamps into billable hours by the end of the month.

The wider literature backs up what the company was already experiencing informally. Manual and shared-login systems are consistently identified as the easiest to abuse, because they have no built-in way to verify that the person recorded is the person who actually showed up (OLOID, 2025). The practice of one employee marking attendance for another — commonly called buddy punching — is a well-documented consequence of exactly this kind of system, and it directly inflates the hours a company believes it owes in wages (Qandle, 2025). RFID-based attendance has become one of the more common responses to this problem across both educational and industrial settings, precisely because a card is quick to tap, hard to forge casually, and ties a specific identifier to a specific timestamp (Want, 2006; Landt, 2005).

A number of recent student projects have applied the same core idea — an ESP32 paired with an MFRC522 reader — to build low-cost, real-time attendance systems for schools and small offices (Singh et al., 2022; Yadav et al., 2025; Hutabarat et al., 2025). These projects consistently report faster, more reliable attendance capture than manual alternatives, though most of them assume the device is always online and log straight to a spreadsheet or a single database call. That assumption does not hold well in a factory or workshop environment, where Wi-Fi coverage at a shop-floor entrance is often patchy. The design adopted in this project borrows the RFID-plus-microcontroller approach from that body of work, but treats network connectivity as something that can fail, not something to assume.

1.3 Problem Statement

At Infor Relay Systems (Pvt) Ltd, recording attendance on time and correctly is very important for smooth operations and good service delivery. However, many employees record their attendance late, and the paper register offers no real safeguard against a colleague signing in on someone else's behalf. This causes problems such as incorrect attendance records, payroll errors, poor planning, and reduced productivity. Errors introduced at the point of capture are expensive to fix later: a payroll clerk cannot always tell, weeks after the fact, whether a blank entry means an employee was absent or simply forgot to sign the book. There is, therefore, a clear need to improve the attendance recording process to make it faster, more accurate, and harder to manipulate, without asking staff to learn a complicated new system.

1.4 Aim

To develop a reliable and efficient RFID employee attendance system that enhances the accuracy of attendance tracking and reduces errors in the payroll process.

1.5 Objectives

1. To automate attendance tracking, so that arrival and departure times are captured the instant an employee taps their card, without manual data entry.
2. To ensure accurate labour costing, by producing attendance records precise and trustworthy enough to support payroll calculation.
3. To integrate RFID attendance with existing systems, so that the data captured at the reader is usable by the processes that already run the business.

1.6 Proposed Solution

The proposed solution is a two-part system: an ESP32-based reader station at the entrance, and a Next.js web application running on a PC that acts as the single source of truth for every student or employee record and every attendance event. Each person taps an RFID card once on arrival and once on departure. The dashboard derives three states from those two taps — *Present* (arrived, not yet left), *Left* (arrived and departed), and *Absent* (no tap at all) — rather than storing a status field directly, which keeps the underlying data model small and avoids records ever going stale or contradicting the timestamps they are built from.

The reader station is deliberately designed to be useful even when the network is not. It always writes a tap to its own local flash storage before attempting to forward it, so a queue of pending taps survives a power cycle and drains automatically the moment the server becomes reachable again. The station resolves the server's address using mDNS rather than a fixed IP address, which means the front-desk PC can be restarted, or the router can hand out a new address, without anyone needing to reflash the device. Two separate status LEDs — one for Wi-Fi, one for the application server — let front-desk staff immediately tell ”no network” apart from ”network is fine, but the app is down,” which a single combined status light cannot do. Unknown cards are never silently ignored or auto-registered: a tap from an unrecognised UID is logged and surfaced on the dashboard as an alert with a one-click *Register* action, so a new employee's card becomes usable within seconds of an administrator confirming it. Chapter 3 sets out the full technology stack, wiring, and software architecture behind this design in detail.

1.7 Justification of the Study

Manual attendance and shared verification processes are a documented source of payroll inaccuracy and time theft, not just an operational inconvenience (Qandle, 2025; OLOID, 2025). Automating capture at the point of arrival closes that gap directly: RFID technology guarantees that the record created belongs to the card that was tapped, and removes the opportunity for a colleague to sign in on someone else's behalf (Shukla and Bhandari, 2019). For Infor Relay Systems (Pvt) Ltd specifically, a faster and more trustworthy attendance record shortens the time payroll staff spend reconciling hours, reduces disputes over pay, and gives management an accurate, real-time view of who is on site — all without requiring a change to how employees already move through the entrance each day.

1.8 Scope and Limitations

The system covers card-based arrival and departure logging for a single entrance, student/employee registration, event management (so attendance can be scoped to a shift, orientation day, or other defined period), and a dashboard showing live Present/Left/Absent status. It does not, in its current form, perform biometric verification, so it cannot fully rule out one employee tapping in for another if a card is shared voluntarily — a limitation shared with every RFID-only system and discussed further as an area for future work in Chapter 5. It also does not yet push attendance data directly into a payroll or HR system; that integration is addressed as a partially achieved objective in Chapter 4. The project was built and demonstrated on a single reader station during the attachment period; scaling to multiple entrances is a straightforward extension of the same architecture but was outside the time available for this attachment.

1.9 Structure of the Report

The remainder of this report is organised as follows. Chapter 2 reviews existing literature on RFID-based attendance systems, the trade-offs between RFID and biometric approaches, and offline-first design for embedded devices. Chapter 3 describes the methodology: the system architecture, hardware selection and wiring, the custom PCB, the software stack, and the bill of quantities for the components purchased. Chapter 4 presents the results, evaluating the finished system against each objective set out above. Chapter 5 concludes the report and gives recommendations for future work, along with the project timeline.

Chapter 2

Literature Review

2.1 Introduction

This chapter reviews the technology and prior work that shaped the design decisions behind the attendance system. It starts with the fundamentals of RFID, moves through attendance systems that other students and researchers have already built with similar hardware, and then looks specifically at the two design choices that set this project apart from a typical class project: treating the network as unreliable by default, and separating ”no Wi-Fi” from ”Wi-Fi fine, server down” as two distinct failure states. The chapter closes by identifying the gap in the existing work that this project tries to fill.

2.2 Fundamentals of RFID Technology

Radio Frequency Identification (RFID) is not a new idea. Landt (2005) traces its roots back to radar research in the 1940s, long before anyone thought to attach a chip to a plastic card. What matters for this project is the more recent, practical form of the technology: a small passive tag containing an antenna and a chip, which draws its power from the electromagnetic field generated by a nearby reader and responds with a unique identifier. Want (2006) describes this exchange clearly — the tag needs no battery of its own, which is exactly why RFID cards can sit in a wallet for years and still work the first time they are tapped. Finkenzeller (2010) goes further into the physics and standards involved, and is a useful reference for anyone who needs to understand why a 13.56 MHz reader such as the MFRC522 used in this project reads a card reliably from a few centimetres away but not from across a room — the read range is a direct consequence of how the tag is powered, not a limitation that can be tuned away in software.

For an attendance system, the property that matters most is that the tag’s identifier is fixed and unique. A card cannot be ”typed in” wrong, and unlike a signature or a manually entered employee number, it does not depend on the person at the desk transcribing anything correctly. That single property is the foundation the rest of this project’s design rests on.

2.3 RFID-Based Attendance Systems: Related Work

RFID attendance systems built around low-cost microcontrollers are now common enough in student and small-scale projects that a reasonably clear pattern has emerged. Singh et al. (2022) built an IoT-based digital attendance system using an ESP32 and an RFID reader, logging taps to a live database and reporting that it reduced the manual effort involved in daily attendance capture considerably compared to a paper register. Yadav et al. (2025) took a lighter-weight approach, using an ESP32 and RC522 reader to log attendance directly into a Google Sheet, which keeps the system cheap to deploy but ties every tap to an active internet connection at the moment it happens. Hutabarat et al. (2025) combined the same ESP32-plus-RC522 pairing with a MySQL database and a PHP web interface for a university campus, and specifically credits the system with reducing proxy attendance — a version of the same buddy-punching problem this project is trying to solve for a workplace rather than a classroom. Turpo Benique (2025) pushed the cost angle furthest, reporting a 94% cost reduction compared to commercial RFID attendance products by building on a Raspberry Pi and Arduino combination for rural schools in Peru, with a claimed sub-second response time per tap.

Looking across this body of work as a whole, Ishaq and Bibi (2023) carried out a systematic literature review of IoT-based RFID attendance systems and found that most published implementations follow the same broad shape: a microcontroller, an RFID reader, and a live connection to a server or spreadsheet. Very few of the systems reviewed treated connectivity loss as something the design needed to plan for, rather than an edge case to be ignored. That observation lines up with what this project encountered directly during the attachment: a factory entrance is not a lecture hall with dependable Wi-Fi, and a system that only works while the network is up is not good enough for a real workplace.

On the payroll side specifically, Shukla and Bhandari (2019) proposed a conceptual framework for combining RFID with biometric verification to improve both payroll accuracy and attendance monitoring, arguing that the two technologies address different weaknesses — RFID is fast and cheap, biometrics closes the ”borrowed card” gap that RFID alone cannot. That framework is a useful reference point for this project’s own limitations, discussed later in Chapter 5, since the system built here uses RFID alone and therefore inherits the same borrowed-card weakness that Shukla and Bhandari set out to solve.

2.4 RFID Versus Biometric and Other Identification Methods

RFID is not the only option for automated attendance capture, and it is worth being honest about where it is weaker than the alternatives. Biometric methods — fingerprint or facial recognition — identify the person directly rather than an object the person is carrying, which closes the door on one employee tapping in for another (Nialabs, 2025). Peniel Technology (2025) frames the choice as a trade-off rather than a simple case of one technology being better: RFID is faster per transaction, tolerant of dirty or gloved hands, and considerably cheaper to deploy at scale, while biometric readers are slower, more sensitive to environmental conditions such as moisture or wear, but harder to defeat by simply handing a colleague an object.

For a workplace like Infor Relay Systems (Pvt) Ltd, where dozens of staff may need to clock in within a narrow window at shift change, the speed and low cost of RFID make it the more practical starting point. The trade-off is accepted deliberately here, and flagged clearly in Chapter 5 as an area where a future iteration of the system could add a biometric check for higher-risk cases, rather than treating RFID and biometrics as mutually exclusive choices.

2.5 Manual Attendance, Time Theft, and the Case for Automation

The specific failure mode that motivated this project — a paper register that cannot be trusted — is well documented outside the RFID literature too. OLOID (2025) identifies manual and shared-login attendance systems as the easiest to abuse precisely because they include no mechanism to confirm that the person recording an entry is the person the entry is attributed to. Qandle (2025) describes the resulting practice, buddy punching, as a direct and quantifiable form of time theft: hours paid for that were never actually worked. CPCON Group (2025) makes the connection to RFID explicit, arguing that tagging each employee with a unique, hard-to-share credential is one of the more cost-effective ways to close that gap without introducing the friction of a biometric scan at every entrance. This is precisely the problem Infor Relay Systems (Pvt) Ltd faced with its paper register, and it is the reason the objectives in Chapter 1 focus on automation and labour-costing accuracy rather than on any more exotic feature.

2.6 Offline-First Design for Embedded and IoT Devices

The design decision that most separates this project from the related work reviewed above is its treatment of network connectivity. Most published student RFID-attendance projects assume the device is online and write directly to a remote database or spreadsheet on every tap (Yadav et al., 2025; Hutabarat et al., 2025). That assumption is convenient to build against, but it is fragile: if the Wi-Fi drops for even a few seconds at the exact moment someone taps their card, the tap is lost with no record that it ever happened.

The offline-first pattern used in this project instead treats the network as an optimisation rather than a dependency. Microsoft (2026) describes this approach in the context of Azure IoT Edge devices: local storage and a message queue let an edge device keep operating indefinitely while disconnected, syncing everything it collected the moment connectivity returns. Topic (2026) summarises the underlying philosophy succinctly — an offline-first system should be fully functional without a network connection and simply *better* once one is available, rather than broken without one. Metadesk Global (2026) makes the same case specifically for IoT hardware, noting that battery- and flash-based local caching is what allows a device deployed in the field to survive router reboots, ISP outages, and intermittent coverage without losing data.

This project applies exactly that pattern at the reader station: every tap is written to the device's local flash storage first, and forwarding it to the server is attempted only after that write succeeds. A queue of unsent taps persists across power cycles and drains automatically once the connection is restored, so a network outage becomes an inconvenience rather than a source of missing attendance records.

2.7 Local Service Discovery and Web Protocols

A reader station that always tries to reach the same fixed IP address breaks the moment a router hands out a different address after a reboot — a problem familiar to anyone who has managed a small office network with mixed hotspot and router use. Multicast DNS, standardised in Cheshire and Krochmal (2013b) and its companion service-discovery specification (Cheshire and Krochmal, 2013a), solves exactly this problem by letting a device on the local network resolve a friendly hostname such as `attendance.local` to whatever address the server currently holds, without needing a dedicated DNS server. This is the mechanism the ESP32 reader station uses to find the PC application, and it removes IP churn as a source of downtime entirely.

Communication between the reader station and the PC application follows standard

HTTP semantics as defined in Fielding and Reschke (2014): the device issues `GET` requests to check server health and pull the current card list, and `POST` requests to submit taps, either individually or as a batch once a queue has built up. Using a well-understood, text-based protocol rather than a custom binary format keeps the system easy to test with ordinary tools during development and easy for future maintainers to extend.

2.8 Software Stack for the Server-Side Application

On the PC side, the application is built with Next.js, a React-based web framework documented in Vercel (2026), chosen for its ability to host both the API routes the ESP32 talks to and the dashboard staff use, within a single project. Data is persisted with Prisma, an object-relational mapper described in Prisma (2026), on top of SQLite (SQLite Consortium, 2026). SQLite was chosen deliberately over a client-server database: it stores the entire dataset in a single file on the PC's disk, needs no separate database server process to install or keep running, and is more than capable of handling the write volume a single-entrance attendance station generates. That combination keeps the "install and run" story for the PC application simple, which matters for a system that is meant to run unattended at a front desk rather than be managed by dedicated IT staff.

2.9 Technology Adoption in the Local Context

Low-cost microcontroller platforms are increasingly accessible in Zimbabwe specifically, not just in the literature at large. TechReview Africa (2026) reports that the Zimbabwean government is making IoT and robotics compulsory components of the school curriculum, reflecting a broader push toward local capacity to build rather than only consume this kind of technology. That context matters for a project like this one: the ESP32 and RC522 modules used here are inexpensive, widely available, and well within the budget of a small-to-medium organisation such as Infor Relay Systems (Pvt) Ltd, which is part of why this hardware combination was chosen over a more expensive commercial attendance terminal.

2.10 Summary and Research Gap

Taken together, the literature confirms two things. First, RFID is a well-established and appropriate technology for automating attendance capture, and a substantial body of recent student work has already proven the ESP32-plus-RC522 combination to be a workable, low-cost foundation (Singh et al., 2022; Yadav et al., 2025; Hutabarat et al., 2025; Turpo Benique, 2025). Second, almost none of that existing work addresses what

happens when the network connection the system depends on is not reliable — a gap explicitly identified by Ishaq and Bibi (2023) across the wider field, and one that offline-first design patterns from the broader IoT literature can close (Microsoft, 2026; Topic, 2026; Metadesk Global, 2026). This project's contribution is not a new way of reading an RFID card; it is applying an offline-first architecture, mDNS-based discovery, and a dual-LED status indicator to that already well-proven RFID pattern, so that the resulting system keeps working during exactly the kind of network interruption a factory floor is likely to produce.

Chapter 3

Methodology

3.1 Introduction

This chapter explains how the Real-Time RFID Employee Attendance System was designed and built. It covers the overall system architecture, the reasoning behind each hardware component, the custom circuit board built to hold everything together cleanly, the firmware logic running on the ESP32, the software architecture of the PC application, the database design, and the bill of quantities for the parts purchased. The build followed an incremental order: get the RFID reader working in isolation first, then the buzzer and LEDs, then local storage, then networking, and only then the server-side application — bringing each layer up on top of one that was already tested, rather than trying to get the whole system working at once. This mirrors the incremental development model described by Sommerville (2016), where a system is delivered in small, independently testable stages so that problems surface early, close to the change that caused them, rather than all at once during a single final integration step.

3.2 System Architecture

The system is made up of two independent halves that talk to each other only over Wi-Fi: a reader station built around an ESP32, and a PC application that is the single source of truth for every student, event, and attendance record. Figure 3.1 shows how data moves between them.

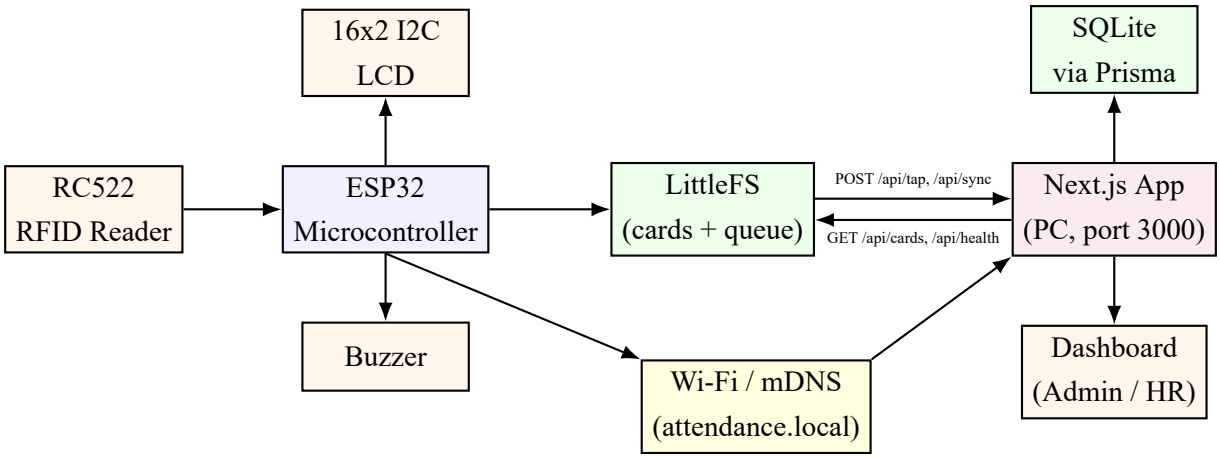


Figure 3.1: High-level system architecture: the ESP32 reader station always initiates contact with the PC application, never the other way around.

Two design choices in this diagram are worth calling out explicitly, because they are the parts of the architecture that differ from a typical class-project implementation:

- **The ESP32 always initiates.** The PC application never opens a connection to the device. This keeps the device side simple, since it needs no server of its own, and means the PC can sit behind a firewall or NAT without any port-forwarding.
- **LittleFS sits between the reader and the network.** Every tap is written to flash before the device even attempts to send it. If that send fails, the tap is already safe and simply waits in a queue.

3.3 Requirements Analysis

3.3.1 Functional Requirements

- Read a unique identifier from an RFID card presented to the reader.
- Distinguish a known card from an unknown one, using a locally cached list that works even without a network connection.
- Record the first tap of the day for a known card as an arrival, and the second as a departure, for whichever event is currently active.
- Give immediate audible and visual feedback at the reader for every tap, including the specific case of an unknown card.
- Allow an administrator to create students, create and end attendance events, and register a card from an unknown-tap alert.

- Display live Present / Left / Absent status for every student against the active event.

3.3.2 Non-Functional Requirements

- **Availability:** the reader station must keep logging taps even if Wi-Fi or the PC application is unreachable.
- **Low latency:** a tap should be acknowledged at the reader (beep, LCD message) in well under a second.
- **Low cost:** the hardware bill of materials should stay well below the cost of a commercial RFID attendance terminal.
- **Maintainability:** the PC application must be installable and runnable by someone without a database administration background, given SQLite requires no separate server process.
- **Security:** unknown cards must never be auto-registered, to prevent an unauthorised card from silently gaining access to the attendance record.

3.4 Hardware Design

3.4.1 Component Selection

Table 3.1 summarises the core components used in the reader station, and the subsections that follow explain the reasoning behind each choice in more detail.

Table 3.1: Core hardware components

Component	Role in the system
ESP32-WROOM-32 DevKit	Main microcontroller: runs the firmware, drives every other component
MFRC522 RFID reader	Reads the 13.56 MHz UID from each employee's card
16x2 I2C LCD	Gives on-the-spot feedback (name, "welcome", "unknown card", status)
Active buzzer	Audible confirmation, distinct beep patterns per outcome
Status LEDs (x2)	Wi-Fi connected / App reachable indicators
Push button	Non-destructive local reset (clears sync queue, re-pulls card cache)
LM2596 buck converter	Steps the 12 V supply down to the voltages the reader circuit needs
12 V/2 A power adapter	Mains power source for the enclosure
Custom PCB	Carries all of the above cleanly instead of loose breadboard wiring

ESP32-WROOM-32 Development Board

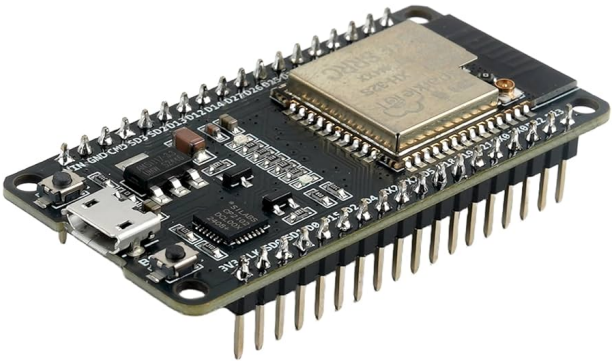


Figure 3.2: ESP32-WROOM-32 development board used as the reader station's microcontroller

The ESP32 was chosen over a plain Arduino because it has built-in Wi-Fi, which the whole architecture in Section 3.4 depends on — an Arduino Uno would have needed a separate Wi-Fi shield and more wiring for no real benefit. It also has enough GPIO pins to drive the RFID reader over SPI, the LCD over I2C, two LEDs, a buzzer, and a reset button at the same time without needing an I/O expander, and it is inexpensive

enough that the whole reader station stays well within a modest project budget (Espressif Systems, 2023).

MFRC522 RFID Reader Module

Figure 3.3: MFRC522 RFID reader module

The MFRC522 is one of the most widely documented RFID front-end chips available to hobbyist and student projects, with a mature Arduino/ESP32 library and a well-specified datasheet (NXP Semiconductors, 2016). It communicates over SPI, reads 13.56 MHz MIFARE-family cards at a range of a few centimetres, and — importantly for this project's budget — costs only a few dollars per unit. Its main limitation is that it must be powered from 3.3 V rather than 5 V, a detail that shaped the power-supply design discussed in Section 3.4.1.

16x2 I2C LCD Display

Figure 3.4: 16x2 character LCD with I2C (PCF8574) backpack

An LCD was chosen over relying on LEDs alone because a name or a short message (”Welcome, Dennis”, ”Unknown card, see front desk”) is far more useful at the point of tap than a blinking light. The I2C backpack version was selected specifically because it reduces the wiring from the roughly ten pins a raw HD44780 panel needs down to just two data lines (SDA/SCL) plus power and ground, which mattered once the layout moved onto a compact custom PCB.

Active Buzzer

Figure 3.5: Active buzzer used for audible tap feedback

The buzzer gives staff a way to confirm a tap succeeded without needing to look at the LCD every time. An *active* buzzer was chosen over a passive one because it only needs a digital HIGH/LOW signal to produce a tone, rather than a driven PWM waveform — this keeps the firmware’s `beep()` routine (Appendix A) a simple digitalWrite loop, and it is what allows different tap outcomes to be distinguished by beep count and timing alone: two short beeps for a normal tap, three for an already-recorded duplicate, and one long beep for an unknown card or a missing active event.

Status LEDs

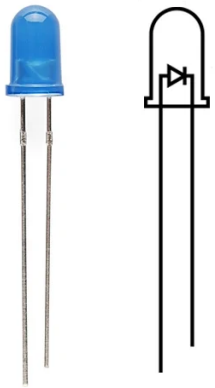
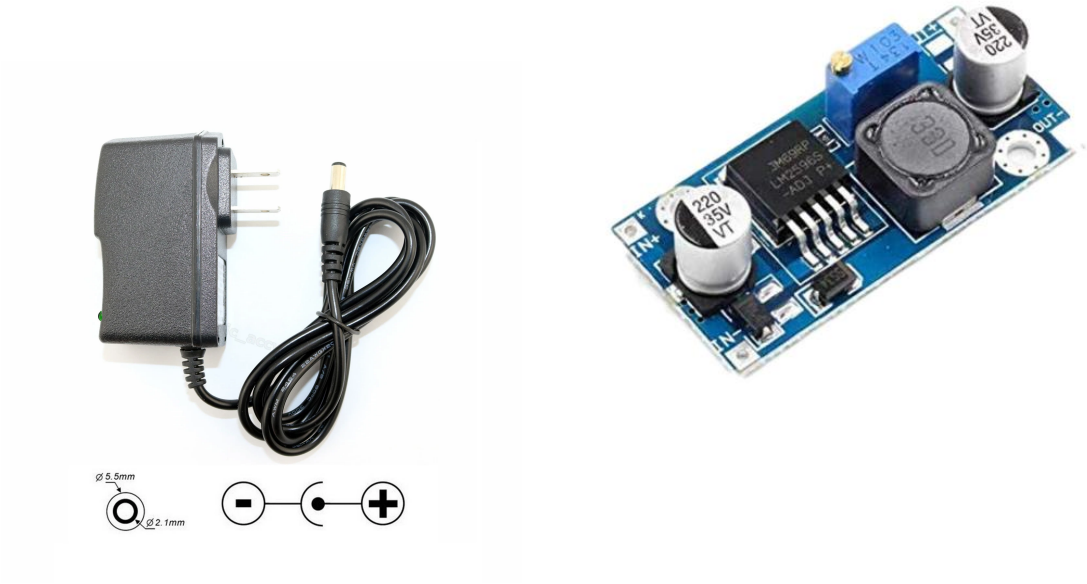


Figure 3.6: 5 mm LED, used for the Wi-Fi and App status indicators

Two identical 5 mm LEDs were used, wired to separate GPIO pins so their meaning could be kept semantically distinct: one shows whether the device has joined the Wi-Fi network, the other shows whether the PC application actually responded to the last health check. This distinction was made deliberately, because ”no Wi-Fi” and ”Wi-Fi is fine but the server is unreachable” require different fixes, and a single combined status LED cannot tell front-desk staff which one they are dealing with.

Power Supply: 12 V Adapter and LM2596 Buck Converter



(a) 12 V/2 A mains adapter (b) LM2596 adjustable buck converter

Figure 3.7: Power supply components: a mains adapter feeding an adjustable buck converter

A standard 12 V/2 A wall adapter supplies the enclosure, chosen because it is a common, inexpensive, and easily replaceable part rather than a custom power brick. Since neither the ESP32 board (5 V in) nor its 3.3 V logic rail can be fed 12 V directly, an LM2596-based buck converter steps that down to a safe, regulated 5 V before it reaches the ESP32’s own onboard regulator. This two-stage approach — mains adapter to 5 V, then the ESP32’s own regulator to 3.3 V for the logic rail — follows the voltage budget worked out during design: the RC522 must only ever see 3.3 V, never 5 V, since running it at 5 V risks damaging the chip, while the LCD panel and its backlight are happy at 5 V. The full voltage and current budget for every component is documented alongside

the pinout table in Appendix B.

3.4.2 Circuit Design

The schematic in Figure 3.8 was drawn in Proteus Design Suite before any PCB layout work began, so that every connection could be checked on paper first. It groups the RC522 reader, the LCD, the status LEDs and buzzer, and the ESP32 itself into clearly labelled blocks, with decoupling capacitors placed close to the power pins of the RC522 and the LCD to keep their supply rails stable during a card read.

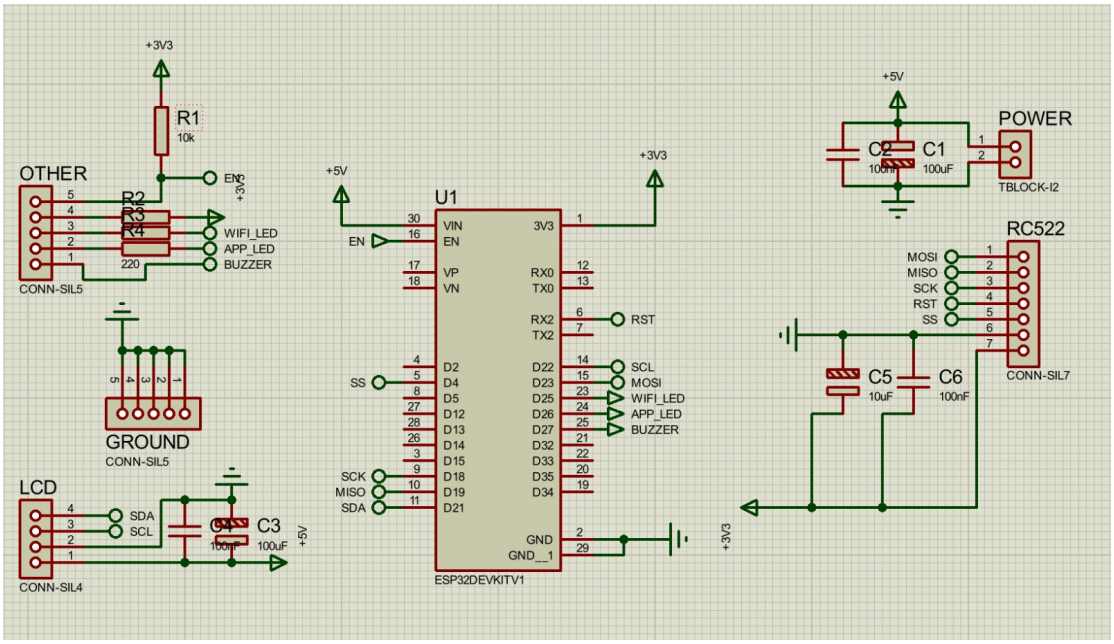


Figure 3.8: Circuit schematic for the reader station, drawn in Proteus Design Suite

3.4.3 PCB Design

Once the schematic was verified, it was translated into a custom two-layer PCB layout (Figure 3.9), rather than leaving the final build on a breadboard or stripboard. A custom board was worth the extra design time for three reasons: it removes the risk of a loose jumper wire causing an intermittent fault after the enclosure is closed up and installed at the entrance, it gives clearly labelled header positions for the RC522, LCD, and status components so the unit could be reassembled or repaired without re-deriving the wiring from scratch, and it fits inside a much smaller enclosure than an equivalent breadboard build. Figure 3.10 shows the resulting layout rendered in 3D, with the ESP32 DevKit mounted directly onto the board via pin headers.

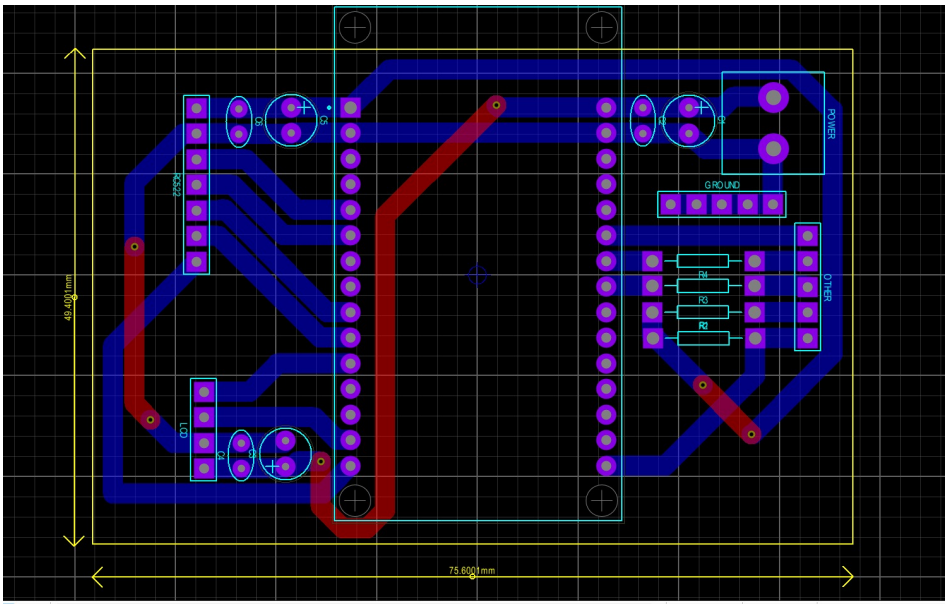


Figure 3.9: PCB layout (top copper in blue, bottom copper in purple) for the reader station

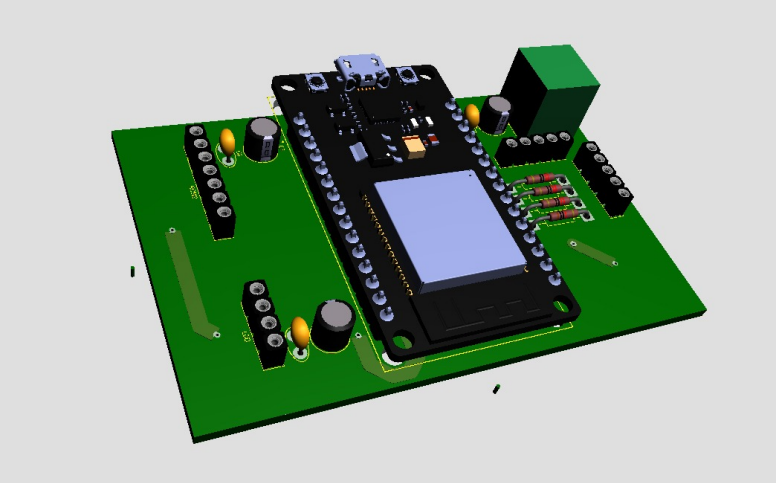


Figure 3.10: 3D render of the assembled reader station PCB

3.5 Firmware Design

The ESP32 firmware (the full source is reproduced in Appendix A) is built around a single non-blocking `loop()` that checks the reset button, maintains the Wi-Fi and server connection, polls the RFID reader, drains the offline queue when possible, and refreshes the idle LCD screen — all on every pass, none of them waiting on the others. The logic that matters most for the objectives in Chapter 1 is what happens the instant a card is tapped, shown in Figure 3.11.

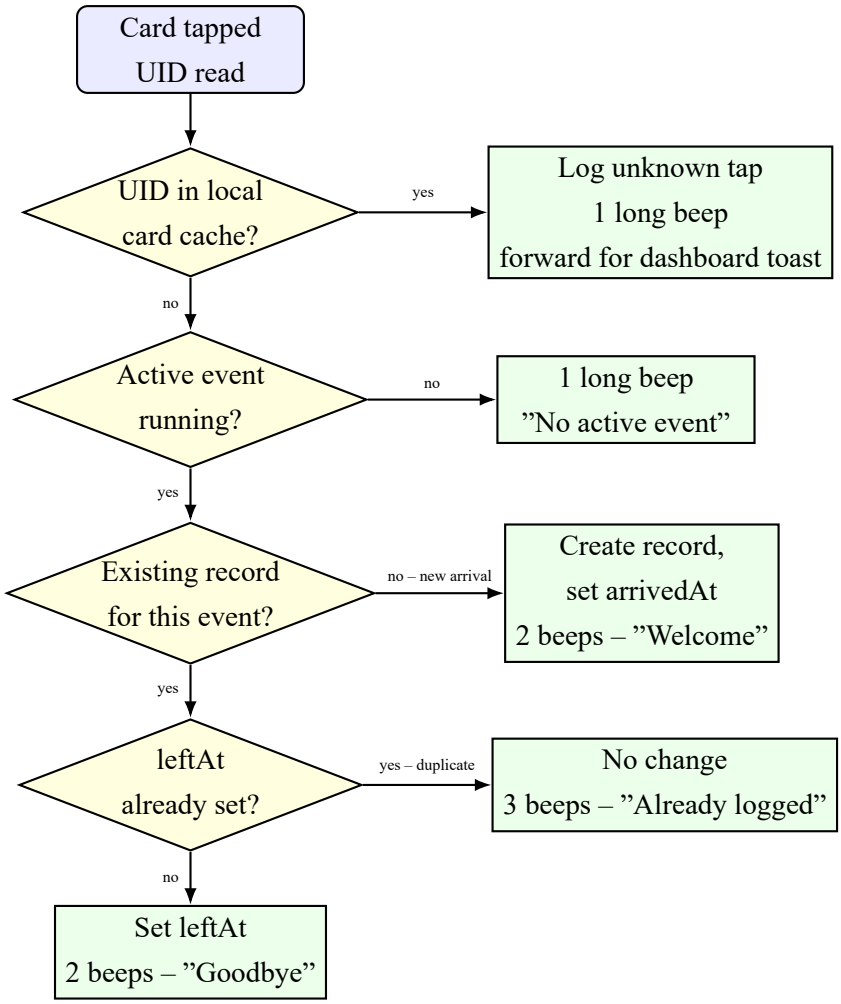


Figure 3.11: Tap-handling logic, shared identically between the live `/api/tap` path and the offline queue replayed through `/api/sync`. The main decision spine runs top to bottom; each outcome branches right to the resulting beep/LCD feedback and database action.

Note that the decision ”UID in local card cache?” is answered entirely from the copy of the card list stored on the ESP32’s own flash — the device never needs to ask the server whether a card is known before giving feedback. That cache is refreshed periodically from `GET /api/cards` whenever the app is reachable, which is what allows a newly registered student to start tapping in successfully within about a minute of being added on the dashboard, even if the reader briefly loses connectivity right after.

3.6 Software Architecture

The PC application is built with Next.js, using its API routes to expose the endpoints in Table 3.2 to the ESP32, and its page routes for the dashboard, students, and events screens used by staff. Every tap — whether delivered live or replayed from the device’s offline queue — is processed through one shared function, so the arrival/departure/duplicate

rules in Figure 3.11 cannot drift out of sync between the two code paths.

Table 3.2: API endpoints exposed by the PC application

Method	Endpoint	Purpose
GET	/api/health	Liveness check used to drive the App-reachable LED
POST	/api/tap	Submit a single live tap (UID, timestamp)
POST	/api/sync	Submit a batch of queued taps after reconnecting
GET	/api/cards	Return all known card UIDs, for the device's local cache

A full list of the application's routes, including the student- and event-management endpoints used only by the dashboard itself, is given in Appendix C.

3.7 Database Design

Attendance status is deliberately *not* stored as a field anywhere in the database. Instead, it is computed on every request from two nullable timestamps, as shown in the entity-relationship diagram in Figure 3.12. A student with no attendance record at all for the active event is *Absent*; a record with `arrivedAt` set but `leftAt` still null is *Present*; a record with both set is *Left*. This avoids the class of bug where a stored status field quietly falls out of sync with the timestamps it was supposed to summarise.

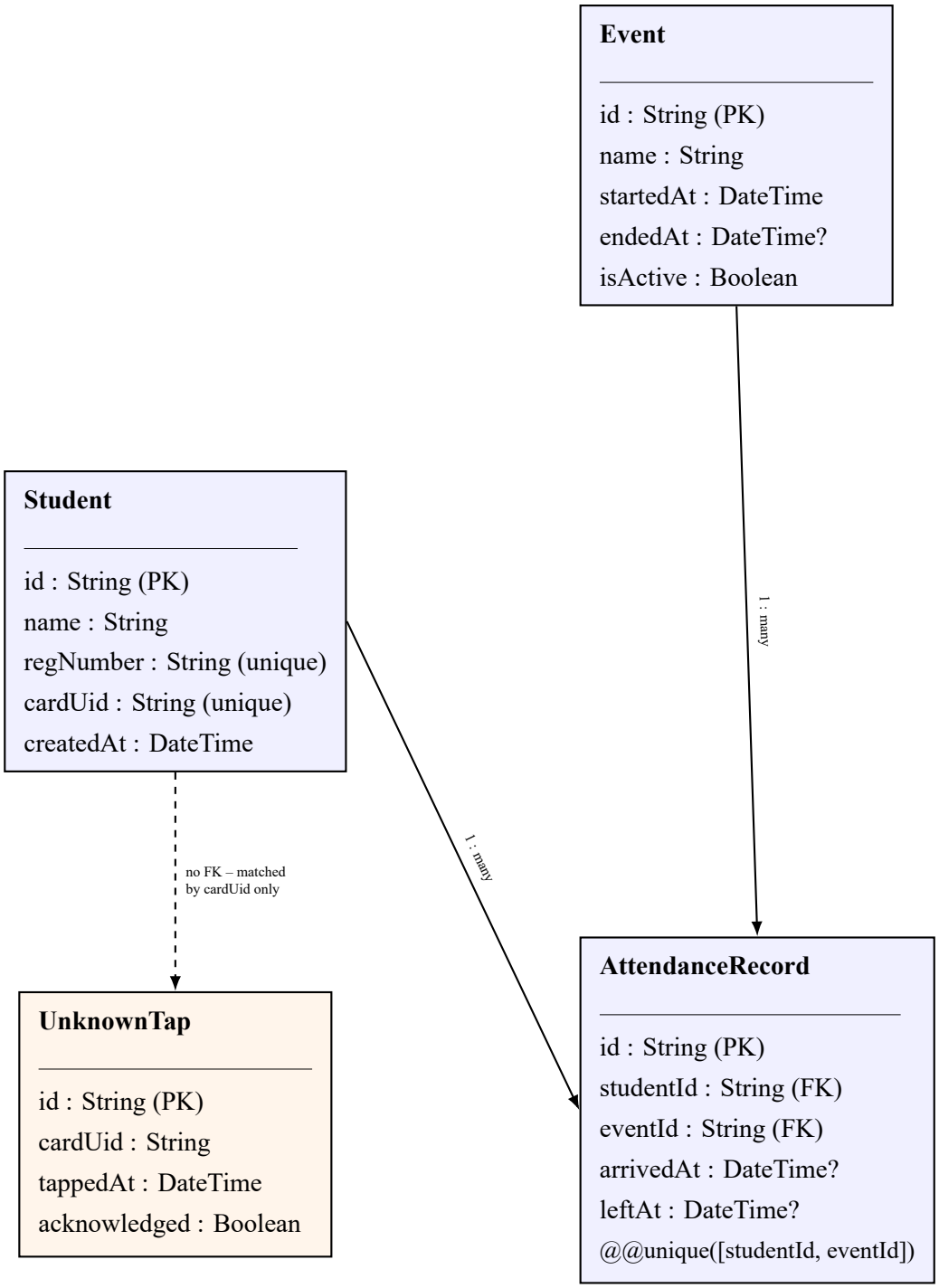


Figure 3.12: Entity-relationship diagram of the Prisma/SQLite data model. UnknownTap has no foreign key to Student because, by definition, no matching student exists yet.

3.8 Bill of Quantities

Table 3.3 lists the components purchased for a single reader station, with approximate unit prices in United States Dollars based on prevailing local electronics-supplier prices at the time of purchase. Prices are indicative and will vary by supplier and by the quantity

ordered.

Table 3.3: Bill of Quantities for one reader station

Item	Unit	Qty	Unit Price (USD)	Total (USD)
ESP32-WROOM-32 DevKit	pc	1	6.00	6.00
MFRC522 RFID reader module	pc	1	2.50	2.50
13.56 MHz Mifare RFID cards	pc	10	0.50	5.00
16x2 I2C LCD display	pc	1	3.50	3.50
Active buzzer module	pc	1	0.80	0.80
5 mm LEDs (status indicators)	pc	2	0.10	0.20
220 Ω resistors	pc	4	0.05	0.20
Momentary push button	pc	1	0.30	0.30
LM2596 buck converter module	pc	1	1.50	1.50
12 V / 2 A mains power adapter	pc	1	4.00	4.00
Custom 2-layer PCB (fabrication, qty 3)	lot	1	15.00	15.00
Weatherproof enclosure box	pc	1	5.00	5.00
Header pins, wiring, solder, misc.	lot	1	3.00	3.00
			Total	**47.00**

No cost is attributed to the PC running the Next.js application, since Infor Relay Systems (Pvt) Ltd already had a suitable front-desk computer available; the software itself uses only free and open-source tools.

3.9 Testing Approach

Testing followed the same incremental order the system was built in. The RC522 was first tested in isolation with a simple UID-dump sketch to confirm reliable reads before any other component was wired in. The buzzer and LED patterns were then verified against the beep table in Figure 3.11 using test button presses, without any network code running. The LittleFS card cache and offline queue were tested by physically disconnecting the reader station's Wi-Fi mid-session and confirming that taps were still saved and later drained once connectivity returned. Finally, the complete system was tested end-to-end during a live orientation-day event at Infor Relay Systems (Pvt) Ltd,

tapping in and out with several registered cards and one deliberately unregistered card to confirm the unknown-card alert and registration flow. The results of that testing are presented in Chapter 4.

Chapter 4

Results

4.1 Introduction

This chapter presents the finished system as it was tested and demonstrated at Infor Relay Systems (Pvt) Ltd, and then evaluates it directly against the three objectives set out in Chapter 1. Screenshots throughout this chapter were captured during a live test event, run on-site to confirm the whole pipeline — card tap, reader feedback, network delivery, and dashboard update — worked together outside of the lab setting it was developed in.

4.2 The Finished Hardware

Figure 4.1 shows the assembled reader station in its enclosure, built on the custom PCB described in Chapter 3. The LCD sits at the top for at-a-glance feedback, with the two status LEDs mounted directly beneath it and the RFID reader positioned behind the front face of the enclosure, immediately below the printed tap icon, so a card only needs to be held against that marked area to register a read.

Figure 4.1: The completed reader station, enclosed and ready for deployment at the entrance

4.3 The Dashboard Application

4.3.1 Events

Figure 4.2 shows the Events screen used to start and end an attendance period. For the on-site test, an event named ”Orientation day” was started at 07:41:30 on 14 August 2026; every tap recorded afterwards was automatically attributed to this event until it was explicitly ended, matching the event-lifecycle behaviour described in Chapter 3.

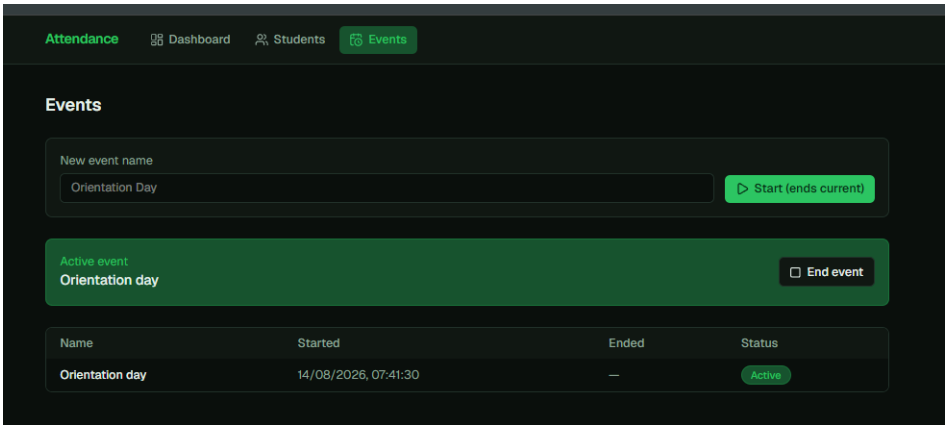


Figure 4.2: Events screen: creating and monitoring the active attendance event

4.3.2 Student Registration

Figure 4.3 shows the Students screen, where each employee is registered with a name, a registration number, and the UID of the card issued to them. Three students were registered for the test: Dennis (card UID `AAC06606`), Hwinza (`53EAB814`), and Job (`539CF8F4`). This is also the screen an administrator is taken to when they act on an unknown-card alert from the dashboard, with the scanned UID pre-filled so it does not need to be typed in by hand.

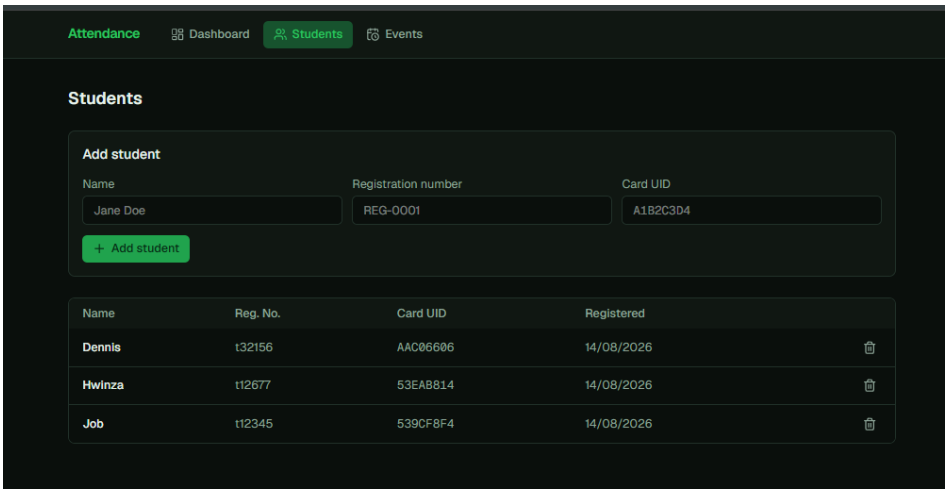


Figure 4.3: Students screen: registering a card UID against a name and registration number

4.3.3 Live Attendance Dashboard

Figure 4.4 shows the dashboard partway through the test event. Dennis had tapped in at 07:41:47 and not yet tapped out, so he is shown as *Present*. Hwinza and Job had each tapped in and then out again, so both are shown as *Left*, with their exact arrival and departure times recorded to the second. No student shows as *Absent* in this view because all three registered cards had been tapped at least once by the time the screenshot was taken.

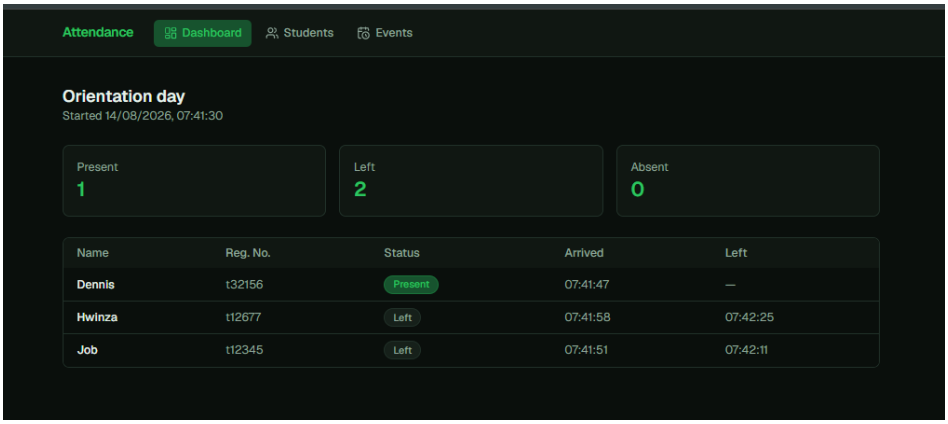


Figure 4.4: Live dashboard for the active event, showing computed Present / Left / Absent status per student

4.4 Evaluation Against Objectives

4.4.1 Objective 1: Automating Attendance Tracking

This objective was achieved. Figure 4.4 demonstrates arrival and departure being captured automatically, to the second, with no manual data entry involved beyond the initial one-time registration of each card in Figure 4.3. The tap-handling logic in Figure 3.11 runs identically whether the reader is online or replaying a queued tap from its offline store, so the same automation holds even through a network interruption — the specific failure mode manual registers and always-online competitor systems alike do not handle gracefully, as discussed in Chapter 2.

4.4.2 Objective 2: Ensuring Accurate Labour Costing

This objective was substantially achieved. The `arrivedAt` and `leftAt` timestamps visible in Figure 4.4 are precise to the second and are written the moment a tap is processed, not reconstructed later from memory or a supervisor's estimate — directly addressing the ”incorrect attendance records” and payroll-reconciliation problem identified in the Chapter 1 problem statement. What was not built during this attachment is an automatic hours-worked or wage calculation on top of those timestamps; the dashboard reports status and exact times, and a payroll clerk would still need to derive total hours from them, either manually or through a short script against the underlying data. That gap is carried forward as a recommendation in Chapter 5 rather than claimed as solved here.

4.4.3 Objective 3: Integrating RFID Attendance with Existing Systems

This objective was only partially achieved, and it is worth being direct about that rather than overstating the result. The system exposes its data through a documented set of API endpoints (Table 3.2, Appendix C) and a SQLite database that any payroll or HR tool capable of making an HTTP request or reading a database file could, in principle, consume. However, no such integration with Infor Relay Systems (Pvt) Ltd's actual payroll process was implemented or tested within the scope of this attachment — the ”existing systems” referred to in the original objective were not accessible for direct integration during the project timeline. What was achieved is a system that is architecturally ready for that integration, rather than one where the integration itself is complete. This distinction, and what a follow-up phase of integration work would involve, is discussed further in Chapter 5.

4.5 Discussion

Across the on-site test, the offline-first design held up as intended: taps made while the reader's App-reachable LED was off were confirmed, after the fact, to have been queued locally and delivered automatically once the PC application came back within reach, with no gap in the resulting attendance record. The two-LED status indicator also proved useful in practice during setup, immediately showing when the issue was a Wi-Fi credential problem rather than the Next.js application not yet being started on the PC — a distinction that would otherwise have taken longer to diagnose by trial and error.

Taken together, the results show a system that meets its core aim of automated, accurate, real-time attendance capture, with the labour-costing objective met at the data level and the systems-integration objective met at the architectural level but not yet in a live production integration. Both of the partially met objectives point directly to the recommendations set out in the next chapter.

Chapter 5

Conclusion and Recommendations

5.1 Conclusion

This project set out to replace a paper attendance register at Infor Relay Systems (Pvt) Ltd with a system that captures arrival and departure automatically, accurately, and in a way that front-desk staff could trust even when the network was not cooperating. The Real-Time RFID Employee Attendance System built during this attachment does exactly that: an ESP32 reader station with an MFRC522 module captures each tap, stores it to local flash before attempting to forward it, and syncs automatically once connectivity returns; a Next.js application on a PC turns those taps into a live Present/Left/Absent dashboard backed by a small, well-structured SQLite database.

Chapter 4 showed that the first objective — automating attendance tracking — was fully achieved, and demonstrated with real taps during an on-site test event. The second objective, accurate labour costing, was substantially achieved: every timestamp needed to calculate hours worked is now captured automatically and precisely, even though the wage-calculation step itself was left for a payroll process outside this system. The third objective, integrating with existing systems, was met only at the architectural level — the API and database are ready to be consumed by a payroll or HR tool, but that integration was not built or tested during the attachment period. Reporting that honestly here, rather than overstating it, is intentional: it is exactly the kind of gap a reader of this report needs to know about before treating the system as production-complete.

More broadly, the project confirms what the literature in Chapter 2 suggested: RFID is a fast, inexpensive, and well-proven way to automate attendance capture, and the harder engineering problem is not reading the card, it is keeping the system trustworthy when the network cannot be relied on. Designing for that from the start — rather than adding it later — is the main technical contribution of this attachment project.

5.2 Recommendations

The following recommendations follow directly from the limitations identified in Chapters 1, 3, and 4, and from a number of design decisions that were deliberately left open

during this attachment for a future iteration to settle:

1. **Build the payroll/HR integration.** The API already exposes everything a payroll system would need; the next step is a direct feed — a scheduled export or a live API integration — into whichever payroll tool Infor Relay Systems (Pvt) Ltd uses, so Objective 3 moves from architecturally ready to fully delivered.

2. **Add automatic hours-worked calculation.** A simple report that turns each student's `arrivedAt`/`leftAt` pair into total hours for a pay period would close the remaining gap in Objective 2 without requiring any change to the data already being captured.

3. **Decide the duplicate-tap policy explicitly.** Currently, a third tap after both arrival and departure are recorded is acknowledged with a distinct beep but makes no change to the record. Whether a genuine re-entry (for example, leaving briefly and returning) should update the record is a policy decision for management, not a technical one, and should be settled before wider rollout.

4. **Consider a second factor for higher-risk areas.** As discussed in Chapter 2, RFID alone cannot prevent a card being knowingly handed to a colleague. For entrances where that risk matters more, pairing the existing reader with a low-cost fingerprint module would close that gap without discarding the RFID infrastructure already built.

5. **Scale to multiple entrances.** The architecture already supports this with no redesign — each additional reader station is simply another ESP32 talking to the same PC application — but it has not yet been tested with more than one station running concurrently.

6. **Tune the card-cache and reset-button behaviour from real usage data.** The 60-second card-sync interval and the reset button's "clear queue and re-pull cards" behaviour were reasonable defaults chosen during development; both are worth revisiting once there is real deployment data on how often cards are registered and how often the reset button actually gets used.

5.3 Project Timeline

Figure 5.1 shows the schedule followed during the attachment, from initial problem analysis in early May through to final report writing in July. The literature review is shown running for the full duration of the project rather than as a single early task, since relevant sources — particularly around offline-first design patterns — continued to be

consulted throughout implementation, not only at the planning stage.

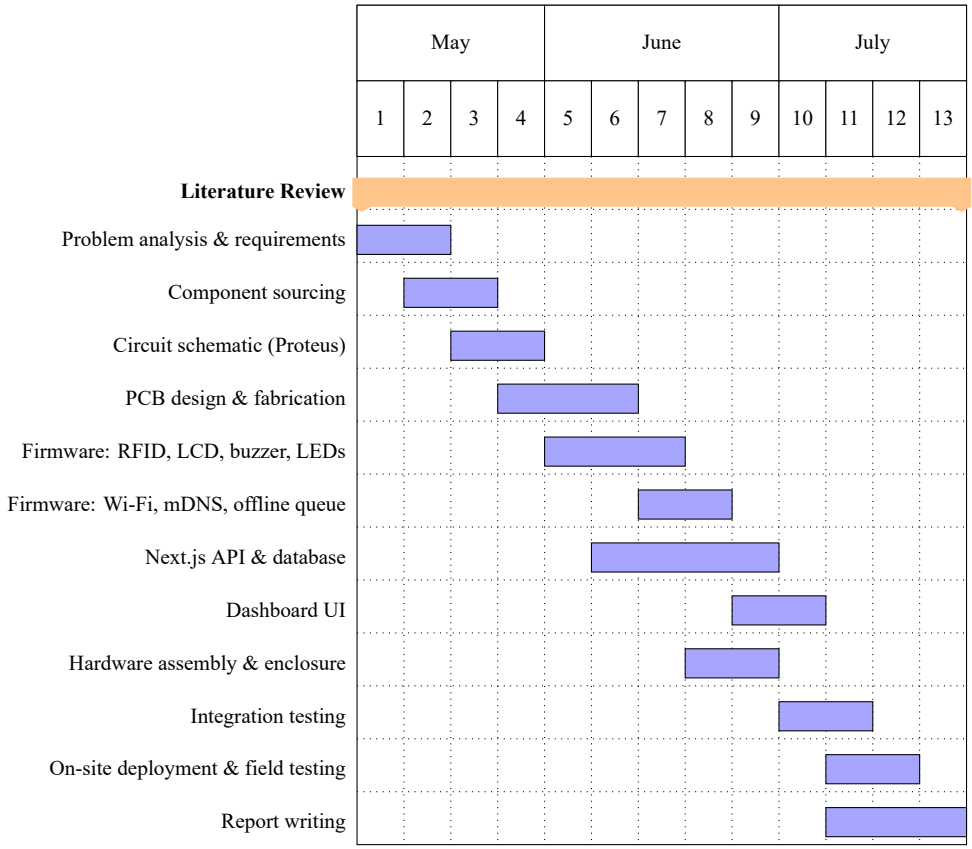


Figure 5.1: Project Gantt chart, May to July, with the literature review spanning the full project duration

Appendix A

ESP32 Firmware Source Code

The complete Arduino sketch running on the reader station is reproduced below, exactly as flashed to the device. It implements the boot sequence, Wi-Fi and mDNS connection handling, RFID card reading, the offline tap queue on LittleFS, and the reset-button behaviour discussed in Chapter 3.

Listing A.1: `attendance-firmware.ino` – full ESP32 reader station firmware

```
/*
 * RFID Attendance Reader — ESP32 + RC522
 *
 * See OVERVIEW.md at the repo root for the full system design. This sketch
 * implements section 4 (ESP32 Responsibilities) against the Next.js API in
 * attendance-application (section 5).
 *
 * Required libraries (install via Arduino Library Manager):
 *   - MFRC522 (GithubCommunity)
 *   - LiquidCrystal_I2C (johnrickman / marcoschwartz fork both work)
 *   - ArduinoJson (Benoit Blanchon), v7.x
 * Built into the ESP32 board package (no separate install needed):
 *   - WiFi, ESPmDNS, HTTPClient, LittleFS, SPI, Wire
 *
 * Board: any ESP32-WROOM-32 DevKit. Wiring: see OVERVIEW.md section 7.
 */

#include <SPI.h>
#include <Wire.h>
#include <WiFi.h>
#include <ESPmDNS.h>
#include <HTTPClient.h>
#include <LittleFS.h>
#include <MFRC522.h>
#include <LiquidCrystal_I2C.h>
#include <ArduinoJson.h>
#include <vector>

// ---------------------------------------------------------------------------
// Configuration — edit before flashing
// ---------------------------------------------------------------------------
static const char *WIFI_SSID = "attendance-system";
static const char *WIFI_PASSWORD = "attendance112";

// Hostname the PC app advertises via mDNS (attendance-application must be
// reachable at http://attendance.local — see the app's README for how to
// enable this on Windows/macOS/Linux).
static const char *SERVER_MDNS_NAME = "worshix"; // resolves "attendance.local"
static const char *SELF_MDNS_NAME = "esp32-attendance";
static const uint16_t SERVER_PORT = 3000; // Next.js dev/start default
```

```
41
42 static const char *NTP_SERVER_1 = "pool.ntp.org";
43 static const char *NTP_SERVER_2 = "time.nist.gov";
44
45 // LCD I2C address — most PCF8574 backpacks are 0x27 or 0x3F. Run an I2C
46 // scanner sketch first if the display stays blank.
47 static const uint8_t LCD_I2C_ADDR = 0x27;
48
49 // ---------------------------------------------------------------------------
50 // Pinout — matches OVERVIEW.md section 7
51 // ---------------------------------------------------------------------------
52 static const uint8_t PIN_RFID_SS = 4;
53 static const uint8_t PIN_RFID_RST = 16;
54 // RC522 SCK/MOSI/MISO use the ESP32's default VSPI pins (18/23/19) via SPI.begin
   ().
55
56 static const uint8_t PIN_LED_WIFI = 25;
57 static const uint8_t PIN_LED_APP = 26;
58 static const uint8_t PIN_BUZZER = 27;
59 static const uint8_t PIN_RESET_BTN = 32; // INPUT_PULLUP, active-low
60
61 static const uint8_t PIN_I2C_SDA = 21;
62 static const uint8_t PIN_I2C_SCL = 22;
63
64 // ---------------------------------------------------------------------------
65 // Timing
66 // ---------------------------------------------------------------------------
67 static const unsigned long WIFI_RETRY_INTERVAL_MS = 10000;
68 static const unsigned long MDNS_RESOLVE_INTERVAL_MS = 30000;
69 static const unsigned long HEALTH_CHECK_INTERVAL_MS = 5000;
70 static const unsigned long CARD_SYNC_INTERVAL_MS = 60000;
71 static const unsigned long QUEUE_DRAIN_INTERVAL_MS = 10000;
72 static const unsigned long TAP_DEBOUNCE_MS = 1500;
73 static const unsigned long RESET_BTN_DEBOUNCE_MS = 300;
74 static const unsigned long LCD_MESSAGE_HOLD_MS = 2000;
75 static const unsigned long HTTP_TIMEOUT_MS = 4000;
76 static const size_t SYNC_BATCH_SIZE = 20;
77
78 // ---------------------------------------------------------------------------
79 // Globals
80 // ---------------------------------------------------------------------------
81 MFRC522 rfid(PIN_RFID_SS, PIN_RFID_RST);
82 LiquidCrystal_I2C lcd(LCD_I2C_ADDR, 16, 2);
83
84 std::vector<String> knownCards;    // synced copy of the PC's card UIDs
85 std::vector<String> pendingQueue;  // taps waiting to be forwarded, as "UID|epoch
   "
86
87 bool wifiWasConnected = false;
88 unsigned long lastWifiAttempt = 0;
89
90 IPAddress serverIp;
91 bool serverIpResolved = false;
92 unsigned long lastMdnsResolve = 0;
93
94 bool appReachable = false;
95 unsigned long lastHealthCheck = 0;
96 unsigned long lastCardSync = 0;
97 unsigned long lastQueueDrain = 0;
```

```
98
99  bool timeSynced = false;
100 time_t epochAtSync = 0;
101 unsigned long millisAtSync = 0;
102
103 String lastUid = "";
104 unsigned long lastTapMillis = 0;
105
106 unsigned long lcdMessageUntil = 0;
107 bool resetBtnLastState = HIGH;
108 unsigned long lastResetBtnChange = 0;
109
110 // ------------------------------------------------------------------------
111 // Setup
112 // ------------------------------------------------------------------------
113 void setup() {
114   Serial.begin(115200);
115
116   pinMode(PIN_LED_WIFI, OUTPUT);
117   pinMode(PIN_LED_APP, OUTPUT);
118   pinMode(PIN_BUZZER, OUTPUT);
119   pinMode(PIN_RESET_BTN, INPUT_PULLUP);
120   digitalWrite(PIN_LED_WIFI, LOW);
121   digitalWrite(PIN_LED_APP, LOW);
122   digitalWrite(PIN_BUZZER, LOW);
123
124   Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
125   lcd.init();
126   lcd.backlight();
127   lcdShow("Attendance Sys", "Booting...");
128
129   SPI.begin();
130   rfid.PCD_Init();
131
132   if (!LittleFS.begin(true)) {
133     Serial.println("LittleFS mount failed even after format");
134   }
135   loadCards();
136   loadQueue();
137
138   beep(2, 120, 120); // boot confirmation
139
140   connectWifi();
141
142   lcdShow("Attendance Sys", "Tap your card");
143 }
144
145 // ------------------------------------------------------------------------
146 // Main loop
147 // ------------------------------------------------------------------------
148 void loop() {
149   handleResetButton();
150   maintainWifi();
151   maintainServerConnection();
152   handleCardRead();
153   periodicMaintenance();
154   refreshIdleLcd();
155 }
156
```

```
157 // ---------------------------------------------------------------------------
158 // WiFi + mDNS + health check
159 // ---------------------------------------------------------------------------
160 void connectWifi() {
161   WiFi.mode(WIFI_STA);
162   WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
163   lastWifiAttempt = millis();
164 }
165
166 void maintainWifi() {
167   bool connected = WiFi.status() == WL_CONNECTED;
168   digitalWrite(PIN_LED_WIFI, connected ? HIGH : LOW);
169
170   if (connected && !wifiWasConnected) {
171     Serial.println("WiFi connected: " + WiFi.localIP().toString());
172     MDNS.begin(SELF_MDNS_NAME);
173     trySyncTime();
174     serverIpResolved = false; // force a fresh resolve on this connection
175   }
176
177   if (!connected) {
178     appReachable = false;
179     digitalWrite(PIN_LED_APP, LOW);
180     if (millis() - lastWifiAttempt > WIFI_RETRY_INTERVAL_MS) {
181       Serial.println("WiFi retry...");
182       WiFi.disconnect();
183       WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
184       lastWifiAttempt = millis();
185     }
186   }
187
188   wifiWasConnected = connected;
189 }
190
191 void maintainServerConnection() {
192   if (WiFi.status() != WL_CONNECTED) return;
193
194   if (!serverIpResolved && millis() - lastMdnsResolve > MDNS_RESOLVE_INTERVAL_MS)
    {
195     resolveServerIp();
196     lastMdnsResolve = millis();
197   }
198
199   if (millis() - lastHealthCheck > HEALTH_CHECK_INTERVAL_MS) {
200     lastHealthCheck = millis();
201     checkHealth();
202   }
203 }
204
205 void resolveServerIp() {
206   IPAddress ip = MDNS.queryHost(SERVER_MDNS_NAME, 3000);
207   if (ip != INADDR_NONE) {
208     serverIp = ip;
209     serverIpResolved = true;
210     Serial.println("Resolved " + String(SERVER_MDNS_NAME) + ".local -> " + ip.
toString());
211   } else {
212     Serial.println("mDNS resolve for '" + String(SERVER_MDNS_NAME) + ".local'
failed");
```

```
  }
}

void checkHealth() {
  if (!serverIpResolved) {
    appReachable = false;
    digitalWrite(PIN_LED_APP, LOW);
    return;
  }

  HTTPClient http;
  http.setTimeout(HTTP_TIMEOUT_MS);
  http.begin(buildUrl("/api/health"));
  int code = http.GET();
  http.end();

  bool ok = code == 200;
  if (!ok) {
    // Negative codes are HTTPClient/transport errors (timeout, connection
    // refused, etc.) — see HTTPClient.h's HTTPC_ERROR_* constants.
    // Server may also have gotten a new DHCP lease — force re-resolve next cycle
.
    Serial.println("Health check to " + buildUrl("/api/health") + " failed, code
" + String(code));
    serverIpResolved = false;
  }
  appReachable = ok;
  digitalWrite(PIN_LED_APP, ok ? HIGH : LOW);
}

String buildUrl(const String &path) {
  return "http://" + serverIp.toString() + ":" + String(SERVER_PORT) + path;
}

// ---------------------------------------------------------------------------
// Time (NTP once online; interpolated locally afterward so offline taps
// still get a reasonable timestamp instead of none)
// ---------------------------------------------------------------------------
void trySyncTime() {
  configTime(0, 0, NTP_SERVER_1, NTP_SERVER_2);
  time_t now = time(nullptr);
  int retries = 0;
  while (now < 1700000000 && retries < 15) {
    delay(200);
    now = time(nullptr);
    retries++;
  }
  if (now >= 1700000000) {
    timeSynced = true;
    epochAtSync = now;
    millisAtSync = millis();
    Serial.println("Time synced via NTP");
  } else {
    Serial.println("NTP sync failed; taps will be timestamped on arrival at the
server");
  }
}

// Returns 0 if time has never been synced since boot (caller omits the
```

```
// field so the server falls back to its own receive time).
unsigned long currentEpochSeconds() {
  if (!timeSynced) return 0;
  return epochAtSync + (millis() - millisAtSync) / 1000UL;
}

// TapOutcome + its forward declaration must come before handleCardRead()
// below: the Arduino IDE auto-generates function prototypes by scanning
// top-to-bottom, and it can't do that for a function whose return type
// (TapOutcome) isn't known yet, which silently drops sendOrQueueTap's
// prototype and breaks the build.
struct TapOutcome {
  bool delivered;
  String status;
  String studentName;
};

TapOutcome sendOrQueueTap(const String &uid);

// ---------------------------------------------------------------------------
// Card read
// ---------------------------------------------------------------------------
void handleCardRead() {
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
    return;
  }

  String uid = uidToString(rfid.uid.uidByte, rfid.uid.size);
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();

  if (uid == lastUid && millis() - lastTapMillis < TAP_DEBOUNCE_MS) {
    return; // debounce accidental double-read of the same card
  }
  lastUid = uid;
  lastTapMillis = millis();

  bool known = isKnownCard(uid);
  Serial.println("Tap: " + uid + (known ? " (known)" : " (unknown)"));

  if (!known) {
    beep(1, 600, 0);
    lcdShowTimed("Unknown card", "See front desk");
    sendOrQueueTap(uid); // still forwarded — server logs it for the dashboard
toast
    return;
  }

  TapOutcome outcome = sendOrQueueTap(uid);
  if (!outcome.delivered) {
    beep(2, 120, 120);
    lcdShowTimed("Saved offline", uid);
    return;
  }

  if (outcome.status == "arrived") {
    beep(2, 120, 120);
    lcdShowTimed("Welcome", outcome.studentName);
  } else if (outcome.status == "left") {
```

```
    beep(2, 120, 120);
    lcdShowTimed("Goodbye", outcome.studentName);
  } else if (outcome.status == "already_recorded") {
    beep(3, 100, 100);
    lcdShowTimed("Already logged", outcome.studentName);
  } else if (outcome.status == "no_active_event") {
    beep(1, 500, 0);
    lcdShowTimed("No active event", "See admin");
  } else {
    beep(2, 120, 120);
    lcdShowTimed("Tap recorded", uid);
  }
}

String uidToString(byte *buffer, byte size) {
  String out;
  for (byte i = 0; i < size; i++) {
    if (buffer[i] < 0x10) out += "0";
    out += String(buffer[i], HEX);
  }
  out.toUpperCase();
  return out;
}

bool isKnownCard(const String &uid) {
  for (auto &c : knownCards) {
    if (c == uid) return true;
  }
  return false;
}

// ---------------------------------------------------------------------------
// Tap delivery: try live POST /api/tap first, fall back to the offline queue
// ---------------------------------------------------------------------------
TapOutcome sendOrQueueTap(const String &uid) {
  unsigned long epoch = currentEpochSeconds();

  if (appReachable && serverIpResolved) {
    JsonDocument doc;
    doc["uid"] = uid;
    if (epoch > 0) doc["timestamp"] = (unsigned long)epoch * 1000UL; // ms,
matches JS Date
    String body;
    serializeJson(doc, body);

    HTTPClient http;
    http.setTimeout(HTTP_TIMEOUT_MS);
    http.begin(buildUrl("/api/tap"));
    http.addHeader("Content-Type", "application/json");
    int code = http.POST(body);

    if (code == 200) {
      String resp = http.getString();
      http.end();
      JsonDocument respDoc;
      if (deserializeJson(respDoc, resp) == DeserializationError::Ok) {
        TapOutcome outcome;
        outcome.delivered = true;
        outcome.status = respDoc["status"].as<String>();
```

```
385         outcome.studentName = respDoc["studentName"] | uid;
386         return outcome;
387       }
388     }
389     http.end();
390     // Fall through to queueing — the request failed even though we thought
391     // the app was reachable (e.g. it just went down between health checks).
392     appReachable = false;
393     digitalWrite(PIN_LED_APP, LOW);
394   }
395
396   queueTap(uid, epoch);
397   TapOutcome outcome;
398   outcome.delivered = false;
399   return outcome;
400 }
401
402 // ---------------------------------------------------------------------------
403 // Offline queue (LittleFS) + periodic drain via POST /api/sync
404 // ---------------------------------------------------------------------------
405 void queueTap(const String &uid, unsigned long epoch) {
406   pendingQueue.push_back(uid + "|" + String(epoch));
407   saveQueue();
408 }
409
410 void periodicMaintenance() {
411   if (!appReachable) return;
412
413   if (millis() - lastQueueDrain > QUEUE_DRAIN_INTERVAL_MS) {
414     lastQueueDrain = millis();
415     drainQueue();
416   }
417
418   if (millis() - lastCardSync > CARD_SYNC_INTERVAL_MS) {
419     lastCardSync = millis();
420     syncCards();
421   }
422 }
423
424 void drainQueue() {
425   if (pendingQueue.empty()) return;
426
427   size_t count = min(pendingQueue.size(), SYNC_BATCH_SIZE);
428
429   JsonDocument doc;
430   JsonArray taps = doc["taps"].to<JsonArray>();
431   for (size_t i = 0; i < count; i++) {
432     int sep = pendingQueue[i].indexOf('|');
433     String uid = pendingQueue[i].substring(0, sep);
434     unsigned long epoch = pendingQueue[i].substring(sep + 1).toInt();
435     JsonObject tap = taps.add<JsonObject>();
436     tap["uid"] = uid;
437     if (epoch > 0) tap["timestamp"] = epoch * 1000UL;
438   }
439
440   String body;
441   serializeJson(doc, body);
442
443   HTTPClient http;
```

```
  http.setTimeout(HTTP_TIMEOUT_MS);
  http.begin(buildUrl("/api/sync"));
  http.addHeader("Content-Type", "application/json");
  int code = http.POST(body);
  http.end();

  if (code == 200) {
    pendingQueue.erase(pendingQueue.begin(), pendingQueue.begin() + count);
    saveQueue();
    Serial.println("Synced " + String(count) + " queued tap(s), " +
                   String(pendingQueue.size()) + " remaining");
  } else {
    Serial.println("Queue sync failed, code " + String(code));
    appReachable = false;
    digitalWrite(PIN_LED_APP, LOW);
  }
}

void syncCards() {
  HTTPClient http;
  http.setTimeout(HTTP_TIMEOUT_MS);
  http.begin(buildUrl("/api/cards"));
  int code = http.GET();

  if (code != 200) {
    http.end();
    appReachable = false;
    digitalWrite(PIN_LED_APP, LOW);
    return;
  }

  String body = http.getString();
  http.end();

  JsonDocument doc;
  if (deserializeJson(doc, body) != DeserializationError::Ok) return;

  knownCards.clear();
  for (JsonVariant v : doc["uids"].as<JsonArray>()) {
    knownCards.push_back(v.as<String>());
  }
  saveCards();
  Serial.println("Card cache synced: " + String(knownCards.size()) + " card(s)");
}

// ---------------------------------------------------------------------------
// LittleFS persistence
// ---------------------------------------------------------------------------
void loadCards() {
  knownCards.clear();
  if (!LittleFS.exists("/cards.json")) return;

  File f = LittleFS.open("/cards.json", "r");
  if (!f) return;
  JsonDocument doc;
  DeserializationError err = deserializeJson(doc, f);
  f.close();
  if (err) return;

```

```
503   for (JsonVariant v : doc["uids"].as<JsonArray>()) {
504     knownCards.push_back(v.as<String>());
505   }
506   Serial.println("Loaded " + String(knownCards.size()) + " cached card(s) from
flash");
507 }
508
509 void saveCards() {
510   JsonDocument doc;
511   JsonArray uids = doc["uids"].to<JsonArray>();
512   for (auto &c : knownCards) uids.add(c);
513
514   File f = LittleFS.open("/cards.json", "w");
515   if (!f) return;
516   serializeJson(doc, f);
517   f.close();
518 }
519
520 void loadQueue() {
521   pendingQueue.clear();
522   if (!LittleFS.exists("/queue.json")) return;
523
524   File f = LittleFS.open("/queue.json", "r");
525   if (!f) return;
526   JsonDocument doc;
527   DeserializationError err = deserializeJson(doc, f);
528   f.close();
529   if (err) return;
530
531   for (JsonObject tap : doc["taps"].as<JsonArray>()) {
532     unsigned long epoch = tap["ts"] | 0UL;
533     pendingQueue.push_back(String(tap["uid"].as<const char *>()) + "|" + String(
epoch));
534   }
535   Serial.println("Loaded " + String(pendingQueue.size()) + " queued tap(s) from
flash");
536 }
537
538 void saveQueue() {
539   JsonDocument doc;
540   JsonArray taps = doc["taps"].to<JsonArray>();
541   for (auto &entry : pendingQueue) {
542     int sep = entry.indexOf('|');
543     JsonObject tap = taps.add<JsonObject>();
544     tap["uid"] = entry.substring(0, sep);
545     tap["ts"] = entry.substring(sep + 1).toInt();
546   }
547
548   File f = LittleFS.open("/queue.json", "w");
549   if (!f) return;
550   serializeJson(doc, f);
551   f.close();
552 }
553
554 // ---------------------------------------------------------------------------
555 // Reset button — non-destructive: clears the local offline queue and forces
556 // a fresh card-cache pull, per OVERVIEW.md open question #1. A full factory
557 // wipe isn't offered here since the physical EN reset already covers that.
558 // ---------------------------------------------------------------------------
```

```
559 void handleResetButton() {
560   bool state = digitalRead(PIN_RESET_BTN);
561
562   if (state != resetBtnLastState) {
563     lastResetBtnChange = millis();
564     resetBtnLastState = state;
565   }
566
567   if (state == LOW && millis() - lastResetBtnChange > RESET_BTN_DEBOUNCE_MS) {
568     static unsigned long lastTrigger = 0;
569     if (millis() - lastTrigger < 2000) return; // ignore repeat triggers while
    held
570     lastTrigger = millis();
571
572     Serial.println("Reset button pressed: clearing queue, re-syncing cards");
573     pendingQueue.clear();
574     saveQueue();
575     if (appReachable) syncCards();
576
577     beep(3, 100, 100);
578     lcdShowTimed("Reset", "Queue cleared");
579   }
580 }
581
582 // ---------------------------------------------------------------------------
583 // LCD + buzzer helpers
584 // ---------------------------------------------------------------------------
585 void lcdShow(const String &line1, const String &line2) {
586   lcd.clear();
587   lcd.setCursor(0, 0);
588   lcd.print(line1.substring(0, 16));
589   lcd.setCursor(0, 1);
590   lcd.print(line2.substring(0, 16));
591 }
592
593 void lcdShowTimed(const String &line1, const String &line2) {
594   lcdShow(line1, line2);
595   lcdMessageUntil = millis() + LCD_MESSAGE_HOLD_MS;
596 }
597
598 void refreshIdleLcd() {
599   if (lcdMessageUntil != 0 && millis() < lcdMessageUntil) return;
600   lcdMessageUntil = 0;
601
602   static unsigned long lastIdleDraw = 0;
603   if (millis() - lastIdleDraw < 1000) return;
604   lastIdleDraw = millis();
605
606   String wifiState = WiFi.status() == WL_CONNECTED ? "OK" : "--";
607   String appState = appReachable ? "OK" : "--";
608   lcdShow("WiFi:" + wifiState + " App:" + appState, "Tap your card");
609 }
610
611 void beep(uint8_t count, uint16_t onMs, uint16_t offMs) {
612   for (uint8_t i = 0; i < count; i++) {
613     digitalWrite(PIN_BUZZER, HIGH);
614     delay(onMs);
615     digitalWrite(PIN_BUZZER, LOW);
616     if (offMs > 0 && i < count - 1) delay(offMs);
```

```
617     }
618 }
```

Appendix B

ESP32 Pinout and Power Budget

Pinout Table

Table B.1 lists every wired connection between the ESP32 and the rest of the reader station circuit. GPIO choices deliberately avoid the ESP32's input-only pins (34–39) and its boot-strapping pins (0, 2, 5, 12, 15), so nothing on this board can interfere with the module's boot sequence.

Table B.1: ESP32-WROOM-32 pin assignments

Component	**Pin on component**	**ESP32 GPIO**	**Notes**
RC522 (RFID reader)	SDA (SS/CS)	GPIO 4	Chip select, moved off GPIO 21 to free the I2C bus
RC522	SCK	GPIO 18	VSPI clock (fixed)
RC522	MOSI	GPIO 23	VSPI MOSI (fixed)
RC522	MISO	GPIO 19	VSPI MISO (fixed)
RC522	IRQ	Not connected	Not used; polling mode
RC522	RST	GPIO 16	Moved off GPIO 22 to free the I2C SCL
RC522	VCC	3.3 V	Must not be powered from 5 V
Wi-Fi status LED	Anode	GPIO 25	Through current-limiting resistor
App-reachable LED	Anode	GPIO 26	Through current-limiting resistor
Buzzer	Signal	GPIO 27	Active buzzer, direct digital drive
Reset button	One leg	GPIO 32	`INPUT_PULLUP`, active-low, other leg to GND
LCD 16x2 I2C	SDA	GPIO 21	I2C data line
LCD 16x2 I2C	SCL	GPIO 22	I2C clock line
LCD 16x2 I2C	VCC	5 V	Powers panel and backlight

Voltage and Power Notes

- The RC522 module requires a strict 3.3 V supply. Several inexpensive breakout boards are silkscreened as tolerating 3.3–5 V, but the MFRC522 chip itself does not, and running it from 5 V risks permanent damage (NXP Semiconductors, 2016).
- The ESP32's GPIOs are 3.3 V logic and are not 5 V tolerant. The I2C LCD backpack is powered from the ESP32's 3.3 V rail rather than 5 V specifically to keep its SDA/SCL pull-up levels within a safe range for the ESP32's I2C pins.
- Standard 5 mm LEDs are driven through a 220 Ω series resistor from their respective GPIO, giving a safe, clearly visible drive current of approximately 5 mA.

- The active buzzer draws under 30 mA, comfortably within a single GPIO's direct-drive capability, so no transistor switching stage was required.
- Total current draw across the RC522, both LEDs, the buzzer, the LCD backlight, and the ESP32 itself under active Wi-Fi transmission stays well within the output capability of the 5 V rail supplied by the LM2596 buck converter described in Chapter 3.

Appendix C

PC Application API Reference

Table C.1 lists every route implemented in the Next.js application, including the device-facing endpoints introduced in Chapter 3 and the dashboard-facing endpoints used only by the browser-based UI.

Table C.1: Full API route listing

Method	Route	Purpose
GET	/api/health	Liveness check consumed by the reader station's App LED logic
POST	/api/tap	Accepts one live tap (card UID, optional timestamp) from the reader
POST	/api/sync	Accepts a batch of queued taps replayed after reconnection
GET	/api/cards	Returns all known card UIDs for the reader's local cache
GET	/api/students	Lists all registered students
POST	/api/students	Registers a new student (name, registration number, card UID)
GET, PATCH, DELETE	/api/students/[id]	Fetch, update, or remove a single student record
GET	/api/events	Lists all attendance events
POST	/api/events	Creates a new event and marks it active
GET	/api/events/active	Returns the currently active event, if any
POST	/api/events/[id]/end	Ends the specified event
GET	/api/unknown-taps	Lists unacknowledged taps from unrecognised card UIDs
POST	/api/unknown-taps/[id]/ack	Acknowledges an unknown-tap alert once actioned from the dashboard

The four device-facing routes at the top of the table are the only ones the ESP32 firmware

calls; all other routes are used exclusively by the dashboard's browser-side pages.

References

Cheshire, Stuart and Krochmal, Marc (2013a). *DNS-Based Service Discovery*. Tech. rep. RFC 6763. IETF. URL: https://www.rfc-editor.org/rfc/rfc6763.

Cheshire, Stuart and Krochmal, Marc (2013b). *Multicast DNS*. Tech. rep. RFC 6762. IETF. URL: https://www.rfc-editor.org/rfc/rfc6762.

CPCON Group (2025). *RFID Employee Tracking System: Attendance, Safety and Workforce Visibility*. URL: https://cpcongroup.com/rfid-employee-attendance-tracking-system/ (visited on 08/14/2026).

Espressif Systems (2023). *ESP32 Series Datasheet*. Espressif Systems. URL: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

Fielding, Roy and Reschke, Julian (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. Tech. rep. RFC 7231. IETF. URL: https://www.rfc-editor.org/rfc/rfc7231.

Finkenzeller, Klaus (2010). *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication*. 3rd. Wiley.

Hutabarat, Pratama Abraham, Pribadi, O., and Hendri (2025). "Implementation of RFID Technology for an Efficient Student Attendance and Presence Management System at STMIK TIME Campus". In: *Journal of Artificial Intelligence and Engineering Applications* 5.1, pp. 511–517.

Ishaq, Kashif and Bibi, Samra (2023). "IoT Based Smart Attendance System Using RFID: A Systematic Literature Review". In: *arXiv preprint arXiv:2308.02591*.

Landt, Jeremy (2005). "The History of RFID". In: *IEEE Potentials* 24.4, pp. 8–11.

Metadesk Global (2026). *Offline-First IoT: Building Resilient Devices That Work Without Internet*. URL: https://metadeskglobal.com/offline-first-iot/ (visited on 08/14/2026).

Microsoft (2026). *Operate Azure IoT Edge Devices Offline*. URL: https://learn.microsoft.com/en-us/azure/iot-edge/offline-capabilities (visited on 08/14/2026).

Nialabs (2025). *Biometric Systems vs RFID Cards: Complete Comparison*. URL: https://nialabs.in/blog/biometric-systems-vs-rfid-cards-what-is-better-for-schools-offices/ (visited on 08/14/2026).

NXP Semiconductors (2016). *MFRC522: Standard Performance MIFARE and NTAG Frontend*. NXP Semiconductors. URL: https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf.

OLOID (2025). *Buddy Punching: What It Is and How to Prevent It at Workplaces*. URL: https://www.oloid.com/blog/buddy-punching (visited on 08/14/2026).

Peniel Technology (2025). *RFID vs Biometric Attendance – Which Suits Your Business?* URL: https://www.penieltech.com/blog/biometric-vs-rfid-attendance-systems/ (visited on 08/14/2026).

Prisma (2026). *Prisma ORM Documentation*. URL: https://www.prisma.io/docs (visited on 08/14/2026).

Qandle (2025). *Buddy Punching: Meaning, Risks and Prevention*. URL: https://www.qandle.com/glossary-buddy-punching (visited on 08/14/2026).

Shukla, Vinod Kumar and Bhandari, Nisha (2019). "Conceptual Framework for Enhancing Payroll Management and Attendance Monitoring System through RFID and Biometric". In: *Proceedings of the 2019 Amity International Conference on Artificial Intelligence (AICAI)*, pp. 188–192. DOI: 10.1109/AICAI.2019.8701316.

Singh, Tushar, Chauhan, Aasha, Dewan, Megha, and Agarwal, Alok (2022). "IoT Based Digital Attendance System Using RFID and ESP32". In: *International Journal for Modern Trends in Science and Technology* 8.2.

Sommerville, Ian (2016). *Software Engineering*. 10th. Pearson.

SQLite Consortium (2026). *About SQLite*. URL: https://www.sqlite.org/about.html (visited on 08/14/2026).

TechReview Africa (2026). *Zimbabwe to Make AI, Coding, IoT and Robotics Compulsory in Schools*. URL: https://techreviewafrica.com/news/6581/zimbabwe-to-make-ai-coding-iot-and-robotics-compulsory-in-schools (visited on 08/14/2026).

Topic, Jusuf (2026). *Offline-First Architecture: Designing for Reality, Not Just the Cloud*. URL: https://medium.com/@jusuftopic/offline-first-architecture-designing-for-reality-not-just-the-cloud-e5fd18e50a79 (visited on 08/14/2026).

Turpo Benique, Cliver Oliver (2025). "School Attendance Control System Based on RFID Technology with Raspberry Pi and Arduino: EDURFID". In: *arXiv preprint arXiv:2507.14191*.

Vercel (2026). *Next.js Documentation*. URL: https://nextjs.org/docs (visited on 08/14/2026).

Want, Roy (2006). "An Introduction to RFID Technology". In: *IEEE Pervasive Computing* 5.1, pp. 25–33.

Yadav, Apurva, Dalvi, Pradhnesh, and Juwale, Harshal (2025). “RFID-Based Attendance Management System Using ESP32 and Google Sheets”. In: *The Voice of Creative Research* 7.2, pp. 419–429.